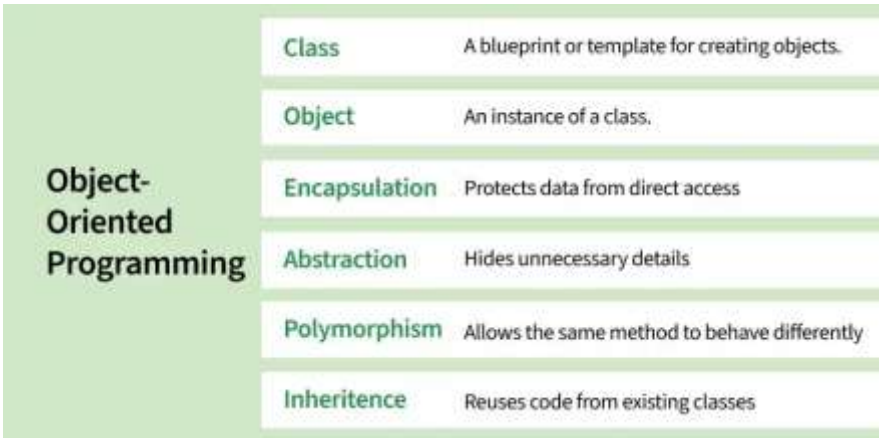
Day – 3 OOPs in Python

Object Oriented Programming empowers developers to build modular, maintainable and scalable applications. OOP is a way of organizing code that uses objects and classes to represent real-world entities and their behavior. In OOP, object has attributes thing that has specific data and can perform certain actions using methods.

- Organizes code into classes and objects.
- Supports encapsulation to group data and methods together.
- Enables inheritance for reusability and hierarchy.
- Allows polymorphism for flexible method implementation.



1. Class:

A class is a collection of objects. Classes are blueprints for creating objects. A class defines a set of attributes and methods that the created objects (instances) can have.

- Classes are created by keyword class.
- Attributes are the variables that belong to a class.
- Attributes are always public and can be accessed using the dot (.) operator. **Example:** Myclass.Myattribute

Example:

```
class Dog:
    species = "Canine"  # Class attribute
    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute
```

2. Objects:

An Object is an instance of a Class. It represents a specific implementation of the class and holds its own data. An object consists of:

- State: represented by the attributes and reflects the properties of an object.
- Behavior: represented by the methods of an object and reflects the response of an object to other objects.
- Identity: gives a unique name to an object and enables one object to interact with other objects.

Example:

```
class Dog:
    species = "Canine"  # Class attribute
    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age  # Instance attribute
# Creating an object of the Dog class
dog1 = Dog("Buddy", 3)
print(dog1.name)
print(dog1.species)
```

Inheritance in Python

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a class (called a child or derived class) to inherit attributes and methods from another class (called a parent or base class).

Example:

```
class Animal:
    def __init__(self, name):
        self.name = name
    def info(self):
        print("Animal name:", self.name)
class Dog(Animal):
    def sound(self):
        print(self.name, "barks")
```

```
d = Dog("Buddy")
d.info()
d.sound()
```

Output:

```
Animal name: Buddy
Buddy barks
```

Benefits Of Inheritance:

- Promotes code reusability by sharing attributes and methods across classes.
- Models real-world hierarchies like Animal -> Dog or Person -> Employee.
- Simplifies maintenance through centralized updates in parent classes.
- Enables method overriding for customized subclass behavior.
- Supports scalable, extensible design using polymorphism.

super() Function:

super() function is used to call methods from a superclass following Python's Method Resolution Order (MRO). In particular, it is commonly used in the child class's __init__() method to initialize inherited attributes. This way, the child class can leverage the functionality of the parent class.

Example:

```
# Parent Class: Animal
class Animal:
    def __init__(self, name):
        self.name = name
    def info(self):
        print("Animal name:", self.name)
# Child Class: Dog
class Dog(Animal):
    def __init__(self, name, breed):
        # Calls constructor based on MRO
        super().__init__(name)
        self.breed = breed
```

```
    def details(self):
        print(self.name, "is a", self.breed)
d = Dog("Buddy", "Golden Retriever")
d.info()      # Parent method
d.details()   # Child method
```

Output:

Animal name: Buddy

Buddy is a Golden Retriever

Types of Inheritance in Python

1.Single Inheritance

Single inheritance enables a derived class to inherit properties from a single parent class, thus enabling code reusability and the addition of new features to existing code.

Example:

```
class Parent:
    def func1(self):
        print("This function is in parent class.")
class Child(Parent):
    def func2(self):
        print("This function is in child class.")
obj = Child()
obj.func1()
obj.func2()
```

Output:

This function is in parent class.

This function is in child class.

2. Multiple Inheritance

When a class can be derived from more than one base class this type of inheritance is called multiple inheritances. In multiple inheritances, all the features of the base classes are inherited into the derived class.

Example:

```
class Mother:
    mothername = ""
    def mother(self):
        print(self.mothername)
class Father:
    fathername = ""
    def father(self):
        print(self.fathername)
class Son(Mother, Father):
    def parents(self):
        print("Father :", self.fathername)
        print("Mother :", self.mothername)
s1 = Son()
s1.fathername = "RAM"
s1.mothername = "SITA"
s1.parents()
```

Output:

```
Father : RAM
Mother : SITA
```

3. Multilevel Inheritance

In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class. This is similar to a relationship representing a child and a grandfather.

Example:

```
class Grandfather:
    def __init__(self, grandfathername):
        self.grandfathername = grandfathername
class Father(Grandfather):
    def __init__(self, fathername, grandfathername):
        self.fathername = fathername
```

```
        Grandfather.__init__(self, grandfathername)
class Son(Father):
    def __init__(self, sonname, fathername, grandfathername):
        self.sonname = sonname
        # Call the constructor of Father
        Father.__init__(self, fathername, grandfathername)
    def print_name(self):
        print('Grandfather name :', self.grandfathername)
        print('Father name :', self.fathername)
        print('Son name :', self.sonname)
s1 = Son('Prince', 'Rampal', 'Lal mani')
print(s1.grandfathername)
s1.print_name()
```

Output:

```
Lal mani
Grandfather name : Lal mani
Father name : Rampal
Son name : Prince
```

4. Hierarchical Inheritance

When more than one derived class are created from a single base this type of inheritance is called hierarchical inheritance. In this program, we have a parent (base) class and two child (derived) classes.

Example:

```
class Parent:
    def func1(self):
        print("This function is in parent class.")
class Child1(Parent):
    def func2(self):
        print("This function is in child 1.")
class Child2(Parent):
    def func3(self):
```

```
        print("This function is in child 2.")
object1 = Child1()
object2 = Child2()
object1.func1()
object1.func2()
object2.func1()
object2.func3()
```

Output:

```
This function is in parent class.
This function is in child 1.
This function is in parent class.
This function is in child 2.
```

5. Hybrid Inheritance

Hybrid inheritance is a combination of more than one type of inheritance. It uses a mix like single, multiple, or multilevel inheritance within the same program. Python's method resolution order (MRO) handles such cases.

Example:

```
class School:
    def func1(self):
        print("This function is in school.")
class Student1(School):
    def func2(self):
        print("This function is in student 1.")
class Student2(School):
    def func3(self):
        print("This function is in student 2.")
class Student3(Student1, School):
    def func4(self):
        print("This function is in student 3.")
obj = Student3()
obj.func1()
```

```
obj.func2()
```

Output:

```
This function is in school.
This function is in student 1.
```